TRABAJO PRÁCTICO FINAL

Calidad de Software A



Integrantes:

- Pezzi, Esteban
- Fernandez Ordoqui,Julian Agustin
- Beltran, Jennifer
- Teruel López, Leonel

11 de noviembre del 2026

Índice

1.Introducción..... 2
2.Objetivos..... 2
2.1. Objetivo General..... 2
2.2. Objetivos Específicos..... 2
3.Desarrollo..... 3
3.1. Testing unitario..... 3
3.1.1. Data Transfer Object (DTO)..... 3
AsociadoDTO..... 3
Fallos encontrados..... 3
3.2. Testing de integración con Base de datos..... 4
3.2.1. DataAccessObject (DAO)..... 4
Fallos encontrados..... 5
3.2.2. BaseDeDatos..... 5
Fallos encontrados..... 6
3.3. Testing de integración con Mocks y Concurrencia..... 6
3.3.1. Dobles de prueba(Mocks)..... 6
3.3.2. Estados..... 6
3.3.3. Concurrencia..... 7
Fallos encontrados..... 7
4.Conclusiones..... 8
5.Apéndice..... 8
Resultados de la ejecución de las suites..... 8
Fallos Detectados..... 9
Resultados de la ejecución de pruebas JUnit..... 9

1.Introducción

Este informe detalla el proceso de testing de software aplicado al sistema de gestión de una clínica. El sistema opera sobre una base de datos relacional MySQL, diseñada para administrar de forma segura los datos de los asociados, implementando patrones arquitectónicos como Objetos de Transferencia de Datos (DTO) para la encapsulación de información y el patrón Singleton para la administración centralizada de la conexión.
Además, el sistema contempla la gestión de estados y concurrencia aplicados a una unidad de Ambulancia.

Para verificar que el sistema cumpla con las validaciones de integridad y los requisitos definidos en la Especificación de Requerimientos de Software (SRS) del módulo de persistencia, se desarrolló una estrategia de pruebas estructuradas en dos niveles arquitectónicos: el testing unitario enfocado en componentes independientes de forma aislada, y el testing de integración para evaluar la interacción real con los recursos externos. También, se incorporó el uso de dobles de prueba (mocks) para simular comportamientos ante la ausencia temporal de implementaciones concretas.

Los resultados de estas pruebas permiten identificar fallos lógicos, errores de mapeo, excepciones de SQL y posibles problemas de sincronización en la ejecución de hilos, brindando información para que se realicen las correcciones necesarias del sistema.

2.Objetivos

2.1. Objetivo General

Verificar la correcta funcionalidad del sistema y comprobar el cumplimiento de los requerimientos de persistencia, integridad de datos y gestión de concurrencia definidos en el documento SRS y las especificaciones del desarrollo.

2.2. Objetivos Específicos

- Desarrollar pruebas unitarias de forma aislada sobre la clase AsociadoDTO para comprobar la correcta asignación, recuperación y las validaciones internas de integridad de sus atributos sin dependencia de servicios externos.
- Aplicar pruebas de integracion sobre los modulos AsociadoDTO, DataAccessObject y BaseDeDatos para comprobar la comunicación real con el motor MySQL.
- Evaluar dentro de la capa de persistencia la correcta ejecución del patrón Singleton en la conexión, la inicialización automática (tablas y datos semillas) y el funcionamiento de las operaciones CRUD parametrizadas.
- Construir pruebas utilizando dobles de prueba (mocks) aprovechando el diseño desacoplado de la interfaz State, simulando los diferentes comportamientos de la máquina de estados.
- Evaluar el flujo completo de la máquina de estados acoplada a la clase Ambulancia y la rutina concurrente RetornoAutomatico.

- Validar la correcta gestión de recursos compartidos, bloques sincronizados y la interacción con hilos independientes de ejecución para detectar cualquier problema de sincronización o fallos de estado introducidos en el sistema.

3.Desarrollo

3.1. Testing unitario

Se verifican de forma aislada los componentes que actúan como contenedores de información antes de interactuar con la capa de persistencia.

El objetivo es asegurar que la lógica interna funcione correctamente sin necesidad de levantar servicios externos. Tomando como base el documento SRS y aplicando principios de diseño por contrato, se asume que las precondiciones ya están garantizadas al recibir los datos. Por lo tanto, las pruebas se orientan a la correcta asignación, recuperación y validación de los datos en memoria ante entradas válidas y casos límite de formato. Para su implementación, se desarrolló una suite de pruebas unitarias utilizando JUnit 4.

3.1.1. Data Transfer Object (DTO)

Manejo de la clase AsociadoDTO, la cual actúa como un contenedor de datos que encapsula la información del asociado. Su principal función es asegurar la integridad de los datos estructurados, implementando las reglas de validación internamente en sus métodos de asignación (setters).

AsociadoDTO

Mediante el uso de tablas de particiones de equivalencia y análisis de valores límite, se diseñó una batería de pruebas para evaluar el comportamiento de la clase ante entradas de datos normales y límites.

- Cadenas de texto: La especificación restringe únicamente los valores nulos, por lo que se comprobó que el sistema reciba y registre correctamente tanto cadenas con datos válidos como cadenas vacías.
- Número de domicilio: Se evaluaron particiones de equivalencias con valores enteros válidos y se sometió el atributo a una prueba de valor límite inferior.

Fallos encontrados

- Metodo: setNumero(int numero)
    - testSetNumero_ValorLimite(): Al evaluar el valor límite inferior especificado (cero), el sistema no registra el dato correctamente y produce un error de ejecución. Se esperaba que el valor fuera aceptado y asignado, pero el código asume incorrectamente que el parámetro debe ser mayor a cero.

Corrección del código previo a la integración

Teniendo en cuenta que en enunciado del trabajo permite “corregir si es necesario el código del item 1” antes de avanzar, se realizó un ajuste en el código fuente de la clase

AsociadoDTO, modificando la aserción para que evalúe “numero >= 0” en vez de “numero > 0”, respetando así la regla definida en el documento SRS.

3.2. Testing de integración con Base de datos

Se verifica la comunicación real entre los módulos AsociadoDTO, DataAccessObject y BaseDeDatos y el motor de base de datos MySQL, sin reemplazar ningún componente por dobles de prueba.

El objetivo es comprobar que los datos se persistan y recuperen correctamente, y que los errores de SQL se propaguen en lugar de ocultarse. Para su implementación se desarrollaron suites con JUnit 4 y el conector mysql-connector-j, ejecutadas contra una base local. Cada prueba parte del mismo estado conocido: un fixture común restaura la base con los 20 datos semilla antes y después de cada prueba, de modo que las pruebas son independientes entre sí. Manteniendo el criterio de diseño por contrato, los parámetros protegidos con assert se consideran precondiciones y no forman parte de los casos de prueba.

3.2.1. DataAccessObject (DAO)

Manejo de la clase DataAccessObject, que expone las operaciones CRUD sobre la tabla Asociados mediante consultas con parámetros.
Mediante tablas de particiones de equivalencia y análisis de valores límite, se diseñó una batería de pruebas para cada operación.

- Unicidad de DNI: El SRS define al DNI como identificador único, regla que no puede verificarse en AsociadoDTO de forma aislada porque la garantiza la restricción UNIQUE de la tabla. Se comprobó que un DNI repetido, tanto al guardar como al actualizar, produzca un error de clave duplicada (código 1062). Sobre la longitud se evaluaron los valores límite de 10 caracteres (válido) y 11 caracteres (error 1406), definidos por la columna VARCHAR(10).
- Guardar: Se comprobó que un asociado válido agregue una fila y que los datos guardados coincidan con los enviados.
- Cargar por id: Se evaluaron los valores límite 1 y 20, un valor intermedio y un id inexistente (21).
- Cargar con límite: Se evaluaron los límites 1 y 20, un valor intermedio y un límite mayor a la cantidad de registros.
- Cargar todos: Se evaluó la tabla con los datos semilla y la tabla vacía.
- Eliminar: Se evaluó un DNI existente y uno inexistente.
- Actualizar: Se comprobó que solo cambie el asociado indicado y que los demás queden igual.

Fallos encontrados

Metodo: cargar(int id)

- testCargar_PrimerId(), testCargar_UltimoId() y testCargar_IdIntermedio(): Al solicitar un asociado existente, el sistema lanza una SQLException (Before start of result set) en lugar de devolverlo. El método lee el ResultSet sin invocar antes a next(), por lo que el cursor todavía se encuentra antes de la primera fila. Se esperaba obtener los datos del asociado.
- testCargar_IdInexistente(): Al solicitar un id que no existe, el sistema lanza una SQLException (Illegal operation on empty result set). Se esperaba obtener null. El método falla en todos los casos, por lo que no es posible cargar un asociado de forma individual, tal como exige el SRS.

Método: actualizar(Asociado asociado)

- testActualizar_ModificaSoloElAsociadoIndicado() y testActualizar_CambiaCalle(): Al actualizar un asociado distinto del primero, el sistema lanza una excepción de clave duplicada (Duplicate entry). Se esperaba que cambiara solo la fila indicada. La causa se encuentra en BaseDeDatos.ejecutarActualizacion(), que fija el id de la sentencia UPDATE en 1 (stmt.setInt(8, 1)), por lo que toda actualización se aplica sobre el asociado con id 1.

El error de clave duplicada aparece porque se intenta escribir en la fila 1 un DNI que ya pertenece a otro asociado. Si el DNI no existe en la tabla, no se produce ningún error y el asociado con id 1 se sobrescribe sin aviso, lo que se verificó actualizando un asociado con DNI 99999998: la fila de Juan State (DNI 12345678) pasó a contener los datos del asociado enviado. Es el fallo de mayor gravedad encontrado, porque altera datos que no corresponden.

3.2.2. BaseDeDatos

Manejo de la clase BaseDeDatos, que administra la conexión con el motor MySQL mediante el patrón Singleton, y ejecuta la inicialización automática de la base de datos.

- Conexión y desconexión: Se comprobó que la conexión se abra y sea válida con el servidor activo, que se cierre correctamente, que cerrarla por segunda vez no produzca errores, y que operar con la conexión cerrada lance una SQLException.
- Singleton: Se comprobó que dos invocaciones de getInstancia() devuelvan el mismo objeto, y que todos los constructores de la clase sean privados.
- Inicialización automática: Se comprobó la existencia de la base Grupo_7 y de la tabla Asociados, la cantidad de datos semilla y el contenido del primer y del último registro, y que una nueva inicialización restaure los datos. También se comprobó que operar sin la base (código 1046) o sin la tabla (código 1146) lance una SQLException.

Fallos encontrados

Las 16 pruebas automatizadas de este módulo aprobaron. El fallo de BaseDeDatos.ejecutarActualizacion() se detecta a través de la operación actualizar del DAO, y se describe en el apartado anterior.

3.3. Testing de integración con Mocks y Concurrencia

Se verifica la clase Ambulancia, la interfaz State con sus implementaciones y la rutina concurrente RetornoAutomatico. Las pruebas se organizaron en tres partes: pruebas atómicas con mocks, pruebas de estados con las implementaciones reales, y pruebas de concurrencia con hilos reales.

3.3.1. Dobles de prueba(Mocks)

Aprovechando que Ambulancia depende de la interfaz State, se reemplazó el estado por un mock construido con Mockito. El mock no ejecuta lógica y registra las llamadas que recibe, lo que permite comprobar a qué método delega Ambulancia sin depender de ninguna implementación concreta.

- **Delegación:** Se comprobó que solicitarMantenimiento, atenderDomicilio, trasladarALaClinica, retornoAutomatico, regresarSinPaciente y volviendoDelTaller invoquen una sola vez al método correcto de State, y que actualicen el indicador de ocupación.
- **Ambulancia ocupada:** Se comprobó que el pedido quede en espera sin invocar al estado, y que continúe al liberarse la ambulancia.
- **Simulación finalizada:** Se comprobó que la operación retorna sin esperar y sin invocar al estado.

Las 13 pruebas de este apartado aprobaron.

3.3.2. Estados

Se evaluó el flujo de cambio de estados acoplado a Ambulancia, con las implementaciones reales de State. Partiendo de cada uno de los seis estados se dispararon los eventos correspondientes, comprobando tanto las transiciones como los eventos que no deben modificar el estado.

3.3.3. Concurrencia

Se evaluó la gestión del recurso compartido con hilos reales. Los hilos se arrancan en simultáneo, su situación se comprueba con Thread.getState(), y toda espera tiene un límite de tiempo, de modo que un bloqueo hace fallar la prueba en lugar de congelarla.

- **Exclusión mutua**: Con cinco hilos pidiendo la ambulancia a la vez se comprobó que solo uno la obtenga, que los demás queden en espera, y que al liberarla terminen todos sin que haya dos hilos dentro al mismo tiempo. También se evaluó un operario compitiendo con un asociado.
- **Espera y despertar**: Se comprobó que un hilo quede en espera con la ambulancia ocupada, y que se despierte al liberarla.
- **Fin de simulación e interrupción**: Se comprobó que los hilos en espera terminen al finalizar la simulación sin invocar al estado, y que un hilo interrumpido termine conservando su bandera de interrupción.
- **RetornoAutomatico**: Como el intervalo de 10 segundos es una constante, se evaluó el ciclo de vida del hilo: si sigue activo o termina según el estado de la simulación.

Las 8 pruebas de exclusión mutua, espera y despertar aprobaron en todas las ejecuciones. Se repitieron de forma consecutiva para descartar resultados inestables.

Fallos encontrados

Metodo: RetornoClinica() de TraslandoPacienteClinicaState

- **testTrasladando_Retorno()**: Al retornar a la clínica desde el estado de traslado, la ambulancia pasa a RegresandoDelTallerState, un estado propio del circuito de mantenimiento. Se esperaba DisponibleState.

Metodo: run() de RetornoAutomatico

- **testSimulacionActiva_ElHiloSigueVivo()**: Con la simulación activa, el hilo termina de inmediato, por lo que nunca intenta el retorno. Se esperaba que siguiera activo realizando intentos periódicos.
- **testSimulacionFinalizada_ElHiloTermina()**: Con la simulación finalizada, el hilo entra al bucle y permanece activo. Se esperaba que terminara de inmediato.

4.Conclusiones

Se ejecutaron 90 pruebas, de las cuales 80 aprobaron y 10 fallaron. Los diez fallos corresponden a cinco defectos del código evaluado, porque varias pruebas fallan por una misma causa. La suite resultó capaz de detectar errores de lógica, excepciones de SQL, errores de persistencia y fallos de estado sin necesidad de reemplazar el código evaluado.

Testing unitario. AsociadoDTO asigna y recupera correctamente sus atributos, pero setNumero rechaza el valor cero que el SRS admite. La regla está implementada con assert, de modo que el fallo sólo se manifiesta con las assertions habilitadas: este tipo de validación depende de la configuración de ejecución y no reemplaza a una validación permanente.

Testing de integración con base de datos. La conexión, el patrón Singleton, la inicialización automática, la unicidad del DNI y las operaciones de guardar, cargar en lote y eliminar funcionan correctamente. Los problemas se concentran en cargar(int id), que no funciona en ningún caso, y en actualizar, que siempre modifica el asociado con id 1. Este último es el defecto más grave encontrado, porque puede alterar los datos de otro asociado sin producir ningún error.

Testing con mocks y concurrencia. El diseño desacoplado de la interfaz State permitió probar Ambulancia de forma aislada, y la delegación resultó correcta. La exclusión mutua, la espera y el despertar de hilos se comportaron correctamente en todas las ejecuciones. Los defectos encontrados no son de sincronización: se trata de una transición incorrecta de la máquina de estados, y de una condición invertida en RetornoAutomatico, que impide que el retorno automático funcione.

5.Apéndice

Resultados de la ejecución de las suites

Suite	Nivel	Pruebas	Aprobadas	Fallidas
TestAmbulanciaConcurrencia	Integración (concurrencia)	8	8	0
TestAmbulanciaEstados	Integración (concurrencia)	15	14	1
TestAmbulanciaMock	Integración	13	13	0
TestAsociadoDTO	Unitario	14	13	1

TestConexionBD	Integración	4	4	0
TestDaoActualizar	Integración	2	0	2
TestDaoCargar	Integración	10	6	4
TestDaoEliminar	Integración	2	2	0
TestDaoGuardar	Integración	1	1	0
TestInicializacionBD	Integración	9	9	0
TestRetornoAutomatico	Integración (Mocks)	3	1	2
TestSingletonBD	Integración	3	3	0
TestUnicidadDNI	Integración	6	6	0
Totales	Integración	90	80	10

Fallos Detectados

Defecto	Ubicación	Pruebas que lo detectan	Resultado obtenido
setNumero rechaza el valor 0	AsociadoDTO.java, línea 87	testSetNumero_ValorLimite	AssertionError
cargar no invoca next()	DataAccessObject.java, líneas 81 a 92	testCargar_PrimerId testCargar_UltimoId testCargar_IdIntermedio testCargar_IdInexistente	SQLException: Before start of result set / Illegal operation on empty result set
UPDATE con id fijó en 1	BaseDeDatos.java, línea 118	testActualizar_ModificaSoloElAsociadoIndicado testActualizar_CambiaCalle	Excepción de clave duplicada: Duplicate entry
Transicion al estado incorrecto	TrasladandoPacienteClinicaState.java, línea 56	testTrasladando_Retorno	RegresandoDelTallerState en lugar de DisponibleState
Condicion del bucle invertida	RetornoAutomatico.java, línea 25	testSimulacionActiva_ElHiloSigueVivo testSimulacionFinalizada_ElHiloTermina	El hilo termina con la simulación activa, y sigue activo con la simulación finalizada

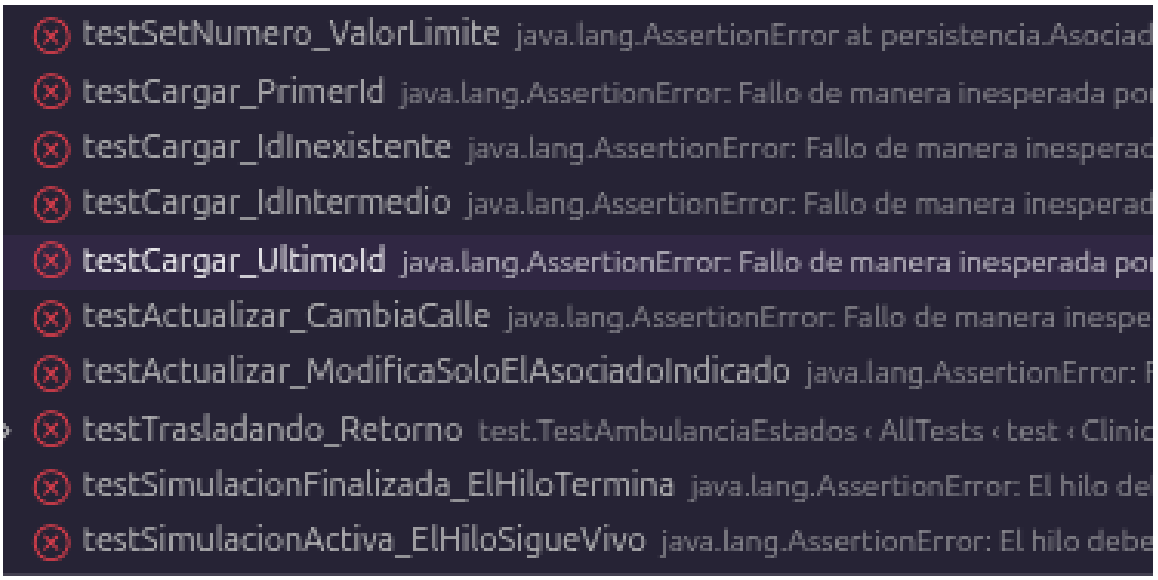
Resultados de la ejecución de pruebas JUnit

Se ejecutó la suite de pruebas **AllTests** mediante JUnit 4 en Eclipse, obteniendo un total de **90 pruebas ejecutadas**, de las cuales **80 fueron exitosas y 10 presentaron fallos**, sin registrarse errores de ejecución.

En las capturas se observan las pruebas superadas, identificadas con íconos verdes, y las pruebas fallidas, señaladas con íconos azules con una X. La barra roja indica que existen pruebas que no cumplieron con los resultados esperados.

Estos resultados permiten identificar posibles defectos en el funcionamiento del sistema y evidencian la utilidad de JUnit para verificar el comportamiento del software

Pruebas que detectaron defectos



Salida de testSetNumero_ValorLimite:

```
java.lang.AssertionError
 at persistencia.AsociadoDTO.setNumero(AsociadoDTO.java:87)
 at test.TestAsociadoDTO.testSetNumero_ValorLimite(TestAsociadoDTO.java:99)
```

Salida de testCargar_PrimerId:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia devolver el asociado con id 1.
 at test.TestDaoCargar.testCargar_PrimerId(TestDaoCargar.java:29)
```

Salida de testCargar_IdInexistente:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia devolver null porque el asociado con id 21 no
 at test.TestDaoCargar.testCargar_IdInexistente(TestDaoCargar.java:88)
```

Salida de testCargar_IdIntermedio:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia devolver el asociado con id 10.
 at test.TestDaoCargar.testCargar_IdIntermedio(TestDaoCargar.java:69)
```

Salida de testCargar_UltimoId:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia devolver el asociado con id 20.
 at test.TestDaoCargar.testCargar_UltimoId(TestDaoCargar.java:49)
```

Salida de testActualizar_CambiaCalle:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia actualizar la calle del asociado con DNI 10112233.
 at test.TestDaoActualizar.testActualizar_CambiaCalle(TestDaoActualizar.java:55)
```

Salida de testActualizar_modificaSoloElAsociadoIndicado:

```
java.lang.AssertionError: Fallo de manera inesperada por un error de SQL cuando debia actualizar el telefono del asociado con DNI 87654321.
 at test.TestDaoActualizar.testActualizar_ModificaSoloElAsociadoIndicado(TestDaoActualizar.java:30)
```

Salida de testTrasladando_Retorno:

```
java.lang.AssertionError: Estado actual incorrecto. expected:[class modelo.modeloDominio.ambulancia.DisponibleState] but was:
[class modelo.modeloDominio.ambulancia.RegresandoDelTallerState]
```

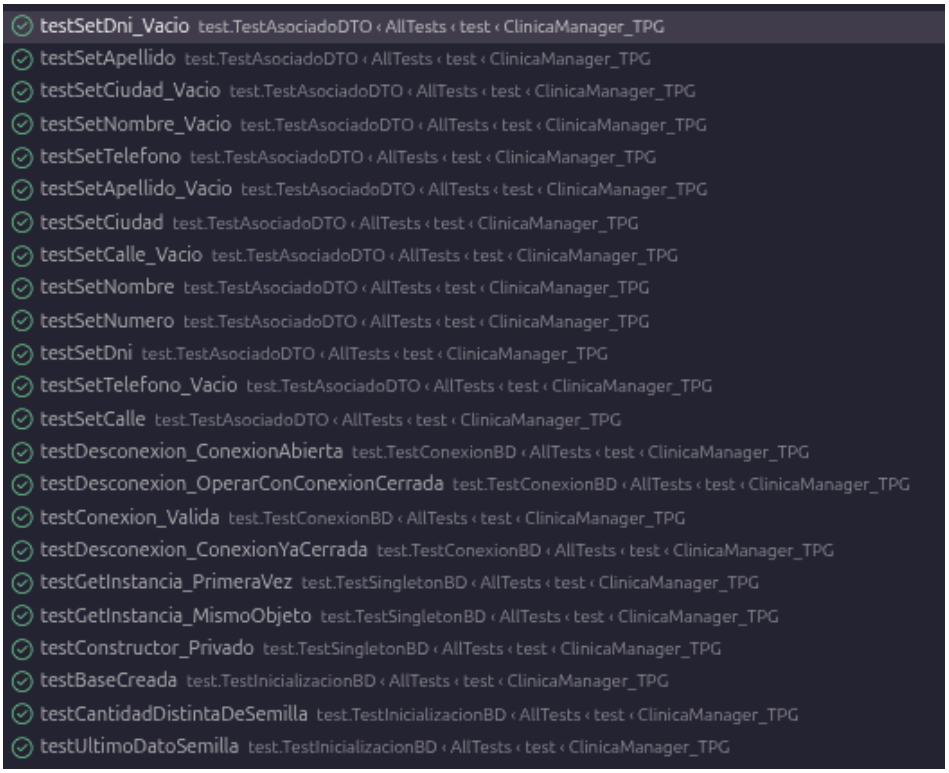
Salida de testSimulacionFinalizada_ElHiloTermina:

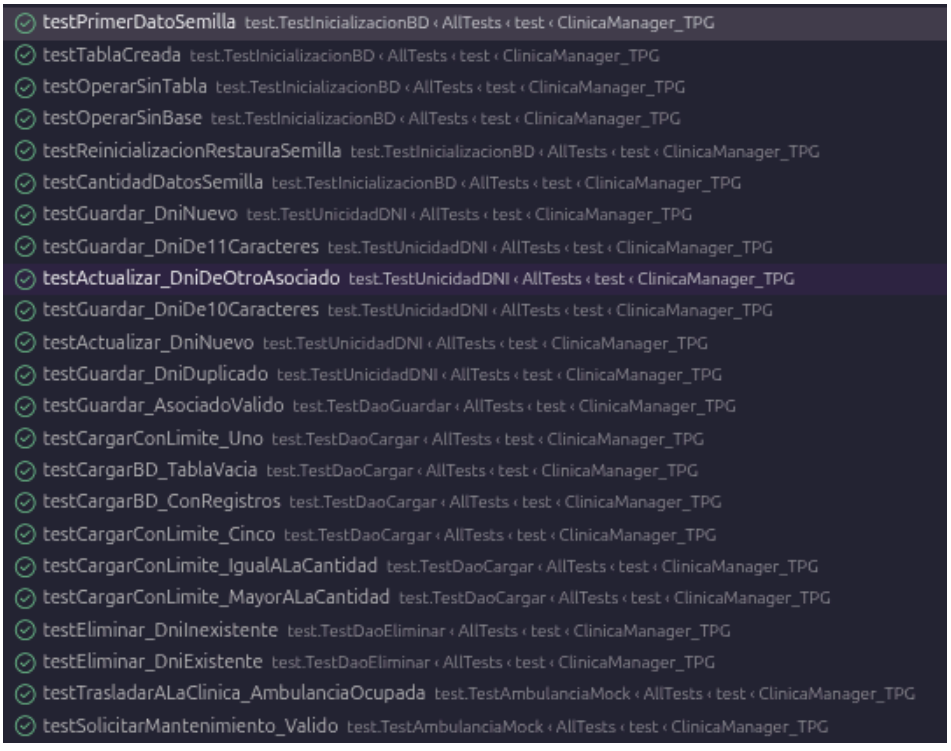
```
java.lang.AssertionError: El hilo deberia terminar si la simulacion ya finalizo.
 at test.TestRetornoAutomatico.testSimulacionFinalizada_ElHiloTermina(TestRetornoAutomatico.java:63)
```

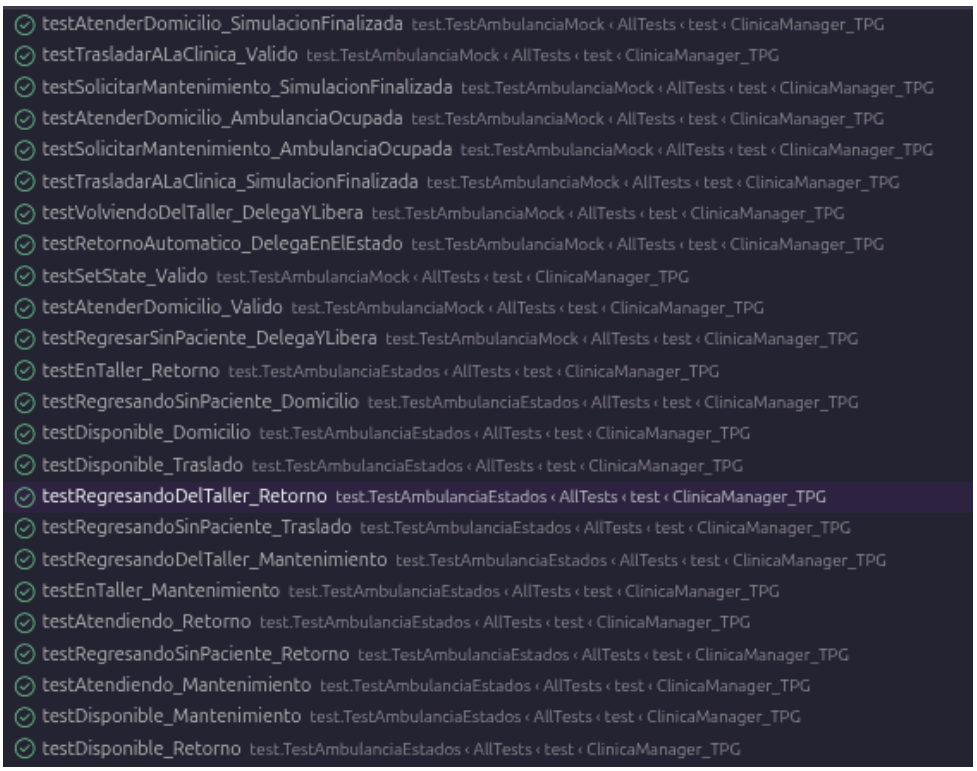
Salida de testSimulacionActiva_ElHiloSigueVivo:

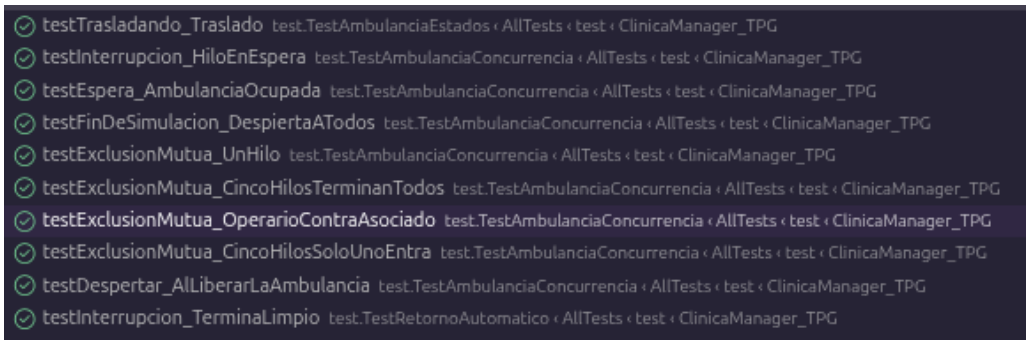
```
java.lang.AssertionError: El hilo deberia seguir vivo mientras la simulacion esta activa.
 at test.TestRetornoAutomatico.testSimulacionActiva_ElHiloSigueVivo(TestRetornoAutomatico.java:36)
```

Pruebas que no encontraron defectos






testAtenderDomicilio_SimulacionFinalizada test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testTrasladarALaClinica_Valido test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testSolicitarMantenimiento_SimulacionFinalizada test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testAtenderDomicilio_AmbulanciaOcupada test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testSolicitarMantenimiento_AmbulanciaOcupada test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testTrasladarALaClinica_SimulacionFinalizada test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testVolviendoDelTaller_DelegaYLibera test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRetornoAutomatico_DelegaEnElEstado test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testSetState_Valido test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testAtenderDomicilio_Valido test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresarSinPaciente_DelegaYLibera test.TestAmbulanciaMock ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testEnTaller_Retorno test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresandoSinPaciente_Domicilio test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testDisponible_Domicilio test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testDisponible_Traslado test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresandoDelTaller_Retorno test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresandoSinPaciente_Traslado test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresandoDelTaller_Mantenimiento test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testEnTaller_Mantenimiento test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testAtendiendo_Retorno test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testRegresandoSinPaciente_Retorno test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testAtendiendo_Mantenimiento test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testDisponible_Mantenimiento test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testDisponible_Retorno test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG


testTrasladando_Traslado test.TestAmbulanciaEstados ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testInterrupcion_HiloEnEspera test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testEspera_AmbulanciaOcupada test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testFinDeSimulacion_DespiertaATodos test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testExclusionMutua_UnHilo test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testExclusionMutua_CincoHilosTerminanTodos test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testExclusionMutua_OperarioContraAsociado test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testExclusionMutua_CincoHilosSoloUnoEntra test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testDespertar_AlLiberarLaAmbulancia test.TestAmbulanciaConcurrencia ‹ AllTests ‹ test ‹ ClinicaManager_TPG
testInterrupcion_TerminaLimpio test.TestRetornoAutomatico ‹ AllTests ‹ test ‹ ClinicaManager_TPG